

Eng 4.7. Introduction to DevSecOps

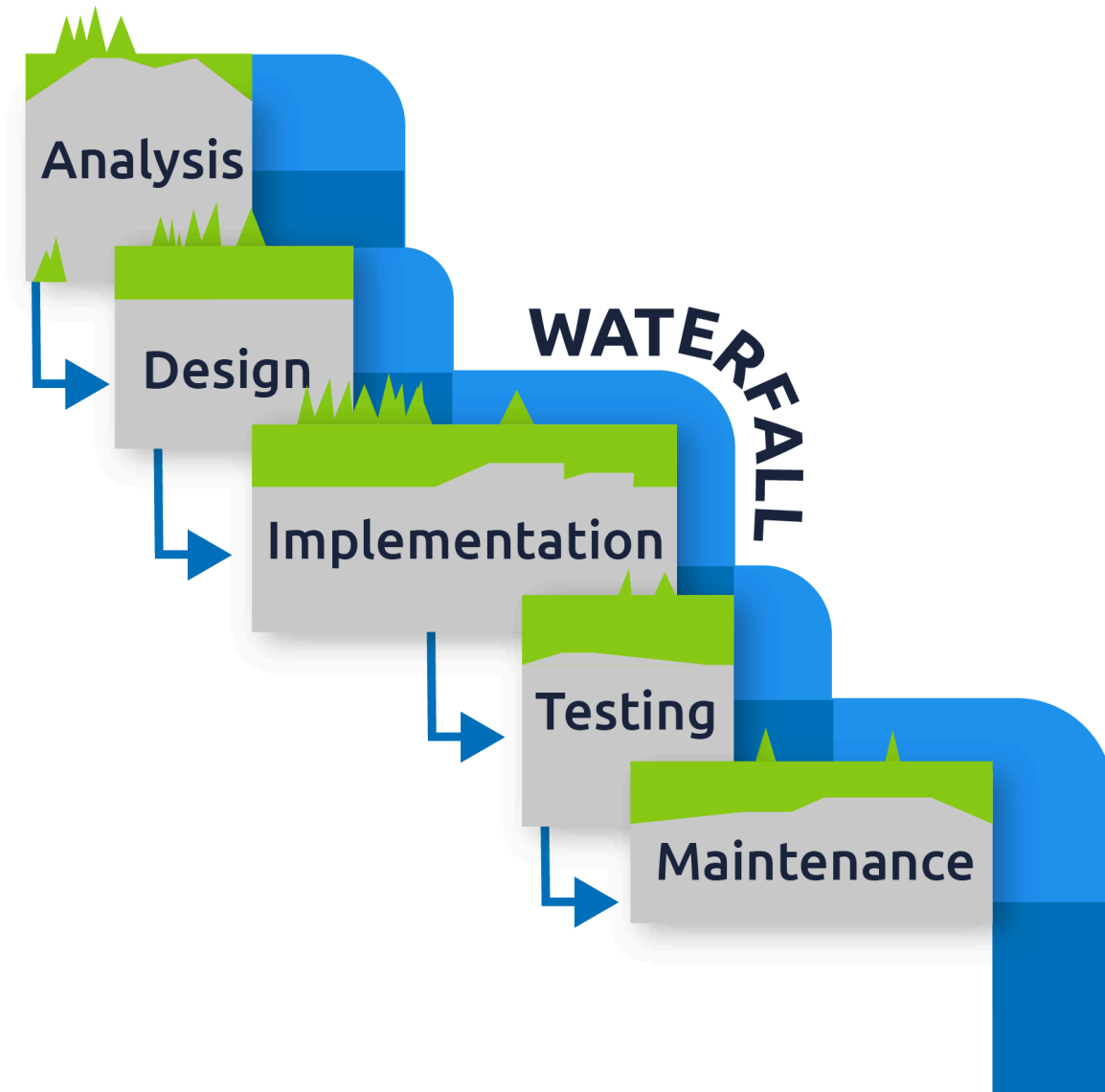
Task 1 - Introduction

This is the first room in a new DevSecOps learning path being developed. The new path will cover:

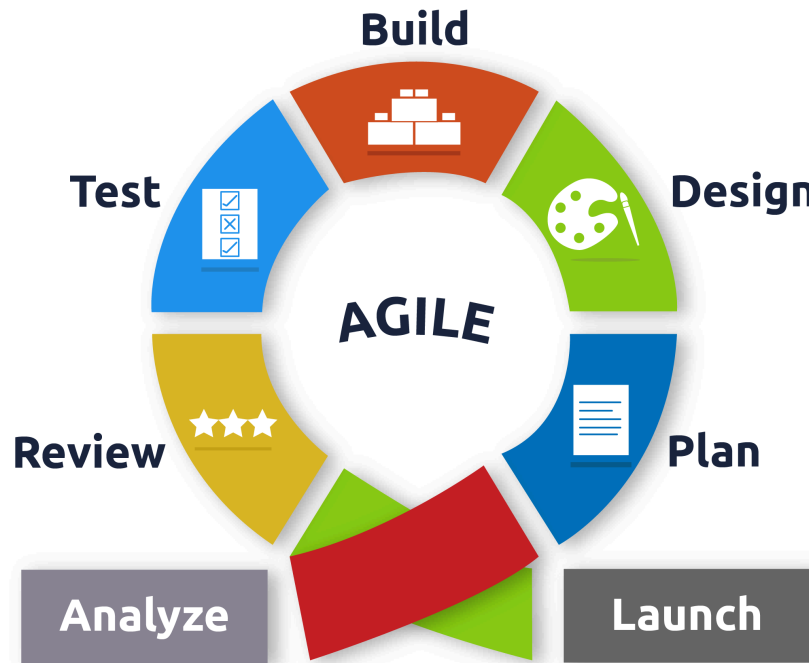
- Introduction to DevSecOps (Secure SDLC, Environments, and Tools)
- Security of the Pipeline (Pipeline Automation, Source Code Security, Automated Code Testing, Dependency Management, CI/CD, and Environment Security)
- Security in the Pipeline (Attacking the Pipeline, Exploiting Vulnerabilities in the Pipeline, Defending the Pipeline)
- Infrastructure as Code (Cloud DevOps, Secret Management, Exploiting Terraform, Exploiting Vagrant, Exploiting Docker)

Task 2 - DevOps Models

Waterfall Model: This is the name given to how project management was approached back in the day (the 70s). The cycle constituted and relied on a *hierarchy*, where every member had a specific responsibility. For example, System admins worked tirelessly to keep everything running smoothly and afloat. Developers build and add as many features as possible, and finally, Quality Assurance (QA) engineers test the system's functionality, ensuring everything works as expected.



Agile Model: Businesses started developing ways that allowed more flexibility and adaptability. Somewhere in early 2000, **The Agile Methodology** was coined. Soon, a manifesto was released: [Agile Manifesto](#)(opens in new tab), emphasising four values for agile development:



DevOps: DevOps builds a philosophy that emphasises building trust and better liaising between developers and other teams (sysadmins, QA, etc.). This helps the organisation align technological projects to business requirements, increasing the impact and value to the business as projects become more efficient and prioritised accordingly. Changes rolled out are usually small and reversible, visible to all teams involved. This ensures better contribution and communication that helps with the pace and an increased competency when delivering work.

- Developers can provide resources to public clouds without depending on IT to provision infrastructure, which in the past led to weeks to months of delays.
- Continuous integration and deployment (CI/CD) processes automatically set up testing, staging, and production environments in the cloud or on-premises. They can be decommissioned, scaled, or re-configured as needed.
- Infrastructure-as-Code (IaC) is widely used to deploy environments declaratively*, using tools like Terraform and Vagrant.
- Organisations can now provision containerised workloads dynamically using automated, adaptive processes
- The declarative approach requires that users specify the end state of the infrastructure - for example, deploy these machines in a running state directly

into an environment, automating the configuration choices throughout the workflow. The software builds it and releases it with no human interaction.

Answer the questions below

What methodology relies on self-organising teams that focus on constructive collaboration?

agile

What methodology relies on automation and integration to drive cultural change and unite teams?

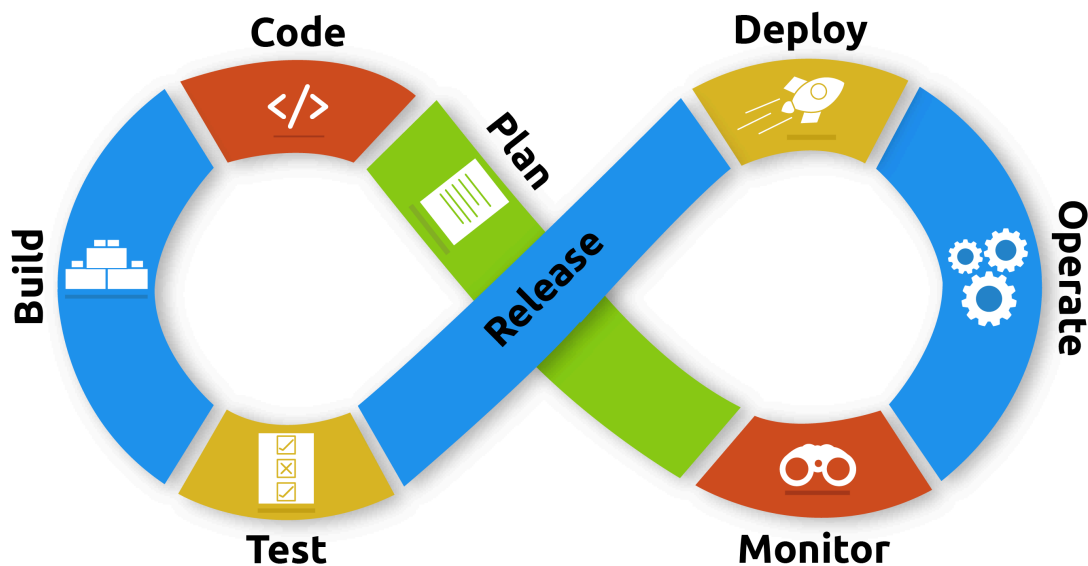
DevOps

What traditional approach to project management led to mistrust and poor communication between development teams? waterfall What does DevOps emphasize?

building trust

Task 3 - The Infinite Loop

How does DevOps work? DevOps is **visualized as an infinite loop, describing all the comprising phases:**



1. **CI/CD** – In the previous task, we mentioned CI/CD (Continuous Integration and Continuous Deployment); CI/CD deals with the frequent merging of code and adding testing in an automated manner to perform checks as new code is pushed and merged. We can test code as we push and merge thanks to a new dynamic and routine in deployment, which takes the form of minor code changes systematically and routinely. Thanks to this change in dynamic, CI/CD helps detect bugs early and decreases the effort of maintaining modular code massively, which introduces reliable rollbacks of versions/code.
2. **INFRASTRUCTURE AS CODE (IaC)** – a way to manage and provision infrastructure through code and automation. Thanks to this approach, we can reuse code used to deploy infrastructure (for example, cloud instances), which helps in consistent resource creation and management. Standard tools for IaC are terraform, vagrant, etc. We will use these tools further in the pathway as we experiment with IaC security.
3. **CONFIGURATION MANAGEMENT** – This is where the state of infrastructure is managed constantly and applying changes efficiently, making it more maintainable. Thanks to this, lots of time is saved, and more visibility into how infrastructure is configured. You can use IaC for configuration management.

4. **ORCHESTRATION** – Orchestration is the automation of workflows. It helps achieve stability; for example, by automating the planning of resources, we can have fast responses whenever there is a problem (e.g., health checks failing); this can be achieved thanks to monitoring.
5. **MONITORING** – focuses on collecting data about the performance and stability of services and infrastructure. This enables faster recovery, helps with cross-team visibility, provides more data to analyze for better root-cause analysis, and also generates an automated response, as mentioned earlier.
6. **MICROSERVICES** – An architecture that breaks an application into many small services. This has several benefits, like flexibility if there is a need to scale, reduced complexity, and more options for choosing technology across microservices. We will look at these in more detail in the DevSecOps pathway.

Answer the questions below

What helps in adding tests in an automated manner and deals with the frequent merging of small code changes?

CI/CD

What process focuses on collecting data to analyse the performance and stability of services?

Monitoring

What is a way to provision infrastructure through reusable and consistent pieces of code?

IaC

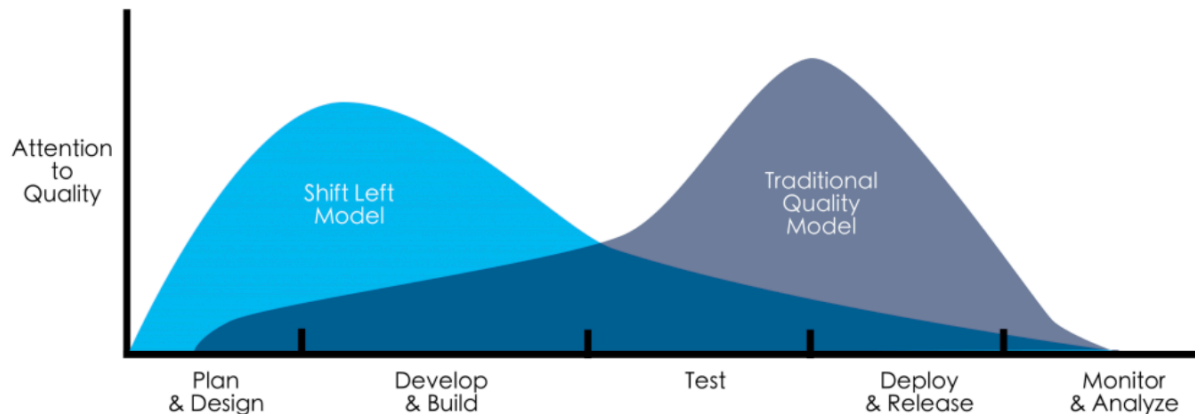
Task 4 - Shifting Left

Shifting Left means that DevOps teams focus on instilling security from the earliest stages in the development lifecycle and introducing a more collaborative culture between development and security.

Implementing security measures during all stages of the development lifecycle (shifting left) rather than at the end of the cycle will ensure the software is

designed with security best practices built in. By detecting security flaws early in development, remediation costs are lower, and there would be no need to roll back changes as they are being addressed on time. This reduces cost, builds trust, and improves the security and quality of the product.

Why are we shifting left



The Shift-Left approach ensures these flaws are caught early by introducing processes from the start. In this fast-paced environment, post-development security reviews of new software versions or analysis of cloud infrastructure configurations become a bottleneck. Even when problems are discovered, there is not enough time to remediate them before the next version or feature is introduced. To keep up with customer needs, they need a fast-paced environment for scaling and growth. Security is at risk of being left behind; instilling security in the beginning and adapting security testing to become flexible and adapted to the development lifecycle increases the chances of addressing things promptly.

Answer the questions below

What term is it used to describe accounting for security from the earliest stages in a development lifecycle?

shift left

What is the development approach where security is introduced from the early stages of a development lifecycle until the final stages?

DevSecOps

Task 5 - DevSecOps: Security Strikes Back

DevSecOps is an approach that relies heavily on automation and platform design that integrates security as a shared responsibility. It is a culture-driven development style that normalises security as a day-to-day operation.

DevSecOps helps bring down vulnerabilities, maximises test coverage, and intensifies the automation of security frameworks. This reduces risk massively, assisting organisations in preventing brand reputation damage, and economic losses due to security flaws incidents, making life easier for auditing and monitoring.

DevSecOps Challenges

Security Silos: Security, development, and operations teams work independently, reducing collaboration.

Lack of Visibility & Prioritization: Too many security findings make it hard to focus on critical risks.

Stringent Processes: Manual security procedures slow down development and deployment cycles.

Answer the questions below

| What DevSecOps challenge can lead to a siloed culture?

Security Silos

| What DevSecOps challenge can affect not prioritizing the right risks at the right times?

Lack of visibility

| What DevSecOps challenge stems from needlessly overcomplicated security processes?

Stringent Processes

Task 6 - DevSecOps Culture

Promote Autonomy: Teams take ownership of security and make decisions independently.

Visibility & Transparency: Security information is shared openly across teams.

Flexibility Through Empathy: Teams understand each other's challenges and collaborate to balance security with speed and business needs.

Answer the questions below

How can you make security scalable so it's not left behind when start ups face hypergrowth or in large corporations?

promote autonomy of teams

How can you support teams in understanding risk and educating on security flaws?

Visibility and Transparency

What are key factors to successfully instill security in the development process by accounting for flexibility?

Understanding and Empathy

Task 7 - Conclusion

SEC3PO, X Fighter Dev, Chewba-QA and S2-A2 have been assigned to discover minerals in nearby planets in the Galaxy. Watto, the Bug, is funding the mission by providing equipment and fuel to carry out this project. They are tasked to set a course and research these planets. Can you guess which Software Development Model they have used in each case to achieve the mission? **Click the [View Site](#) button on the top right, and** look at the static site attached to this task. **Can you figure out which approach was taken in each comic snippet? To view the comic again, click the [View Site](#) button again.**

Background

Mission: Travel to the planet with the least amount of risk possible.

Fuel: Accounts for 130.000 Light Years of travel

Models: Waterfall, Agile & DevOps

Planet	Distance from Spaceship
Testooine	125.000 Light Years
Hackboo	100.000 Light Years
TryHothMe	80.000 Light Years
Dagobug	60.000 Light Years

Comic 1: Some tests have passed to go to Testooine, which was the initial decision by x fighter dev. It is decided that it is the next planet to visit.

Comic 2: Some tests indicated that it is a high risk to travel to Testooine; first, some tests have passed for Naboo, but S2-D2 has decided the course should be changed to Hoth.

Comic 3: The initial decision based on analysis by X fighter Dev is to travel to Hoth. Costs were questioned, and although Hoth is one of the closest, SEC3PO has analysed the trajectory and has set new parameters for the tests. Chewba-QA has concluded to change orbit to Dagobah based on further tests.

Answer the questions below

| What Software Development Model did the team in Comic 1 follow?

Waterfall

| What Software Development Model did the team in Comic 2 follow?

Agile

| What Software Development Model did the team in Comic 3 follow?

DevOps

| What is the flag?

THM{ONE_TWO_THREE}

TRYHACKME COMICS

THM{ONE_TWO_THREE}

